

Question: Create a **C# Console Application** for a company's **Employee Management System** using **OOP** principles.

Requirements:

1. Define an Interface (IEmployee)

- Method: `void DisplayDetails();`

2. Create a Base Class (Employee)

- Properties:
 - `string Name { get; set; }`
 - `int EmployeeID { get; set; }`
 - `double Salary { get; set; }`
- Constructor:
 - Initializes `Name`, `EmployeeID`, and `Salary`.
- Virtual Method:
 - `public virtual void CalculateBonus()`

3. Create Derived Classes (Manager and Developer)

- Both classes inherit from `Employee` and implement `IEmployee`.
- Override `CalculateBonus()` method:
 - `Manager` gets **20% bonus** of salary.
 - `Developer` gets **10% bonus** of salary.
- Implement `DisplayDetails()` method from `IEmployee`.

4. In the Main Method (Program.cs)

- Create objects for `Manager` and `Developer` using Interface reference and initialize data using a constructor. (`IEmployee = new Manager..`)
- Call `CalculateBonus()` and `DisplayDetails()` for both.

Expected Output (Example):

```
Manager: John Doe (ID: 101)
Salary: $8000
Bonus: $1600
```

```
Developer: Alice Smith (ID: 102)
Salary: $6000
Bonus: $600
```

Question: Create a **C# Console Application** that simulates an **Employee Payroll System** using **Partial Classes**, **Enums**, and **Extension Methods**.

Requirements:

1. Define an **EmployeeType** Enum

Represents different employee roles:

```
public enum EmployeeType { Manager, Developer, Intern }
```

2. Create a Partial Class (**Employee**)

- **First Part (Employee_Details.cs):**

- Properties:

- `string Name`
- `int ID`
- `double Salary`
- `EmployeeType Type` (use the enum)

- Constructor to initialize these properties.

- **Second Part (Employee_Methods.cs):**

- Method `void DisplayDetails()` to print employee details.

3. Create an Extension Method (**EmployeeExtensions**)

- Define a method `public static double CalculateAnnualSalary(this Employee emp)`
- Returns `emp.Salary * 12`

4. In the **Main** Method (**Program.cs**)

- Create an `Employee` object with a specific `EmployeeType`.
- Call `DisplayDetails()` to show employee information.
- Use the **extension method** to calculate and display the **annual salary**.

Expected Output (Example):

```
Employee: John Doe (ID: 101)
```

```
Role: Developer
```

```
Monthly Salary: $5000
```

```
Annual Salary: $60000
```

Question: Create a **C# Console Application** that demonstrates the use of **delegates** for performing arithmetic operations.

Requirements:

1. Define a Delegate (**ArithmeticOperation**)

- `delegate int ArithmeticOperation(int a, int b);`

2. Create a **Calculator** Class

- Define four static methods:
 - `Add(int a, int b)` → Returns sum
 - `Subtract(int a, int b)` → Returns difference
 - `Multiply(int a, int b)` → Returns product
 - `Divide(int a, int b)` → Returns quotient (handle division by zero)

3. In the **Main** Method (**Program.cs**)

- Declare a delegate variable of type `ArithmeticOperation`.
- Assign it different methods (`Add`, `Subtract`, `Multiply`, `Divide`).
- Call the delegate for each operation and display the results.

Expected Output (Example):

makefile

CopyEdit

Addition: 10 + 5 = 15

Subtraction: 10 - 5 = 5

Multiplication: 10 * 5 = 50

Division: 10 / 5 = 2

Question: Create a **C# Console Application** that implements a **Generic Data Storage System** using **Generics** and **Indexers**.

Requirements:

1. Create a Generic Class (**Storage<T>**)

- **Private List** to store items of type `T`.
- **Methods:**
 - `void AddItem(T item)` → Adds an item to the storage.
 - `void RemoveItem(T item)` → Removes an item if it exists.
 - `void DisplayItems()` → Displays all stored items.
- **Indexer:**
 - `public T this[int index]` → Allows access to stored items using an index.

2. In the **Main** Method (**Program.cs**)

- Create an instance of **Storage<T>** for different data types:
 - Store **int** values (e.g., product IDs).
 - Store **string** values (e.g., employee names).
 - Store **double** values (e.g., product prices).
- Add, remove, and display items from each storage.
- Use the **indexer** to retrieve specific items by index.

Expected Output (Example):

yaml

CopyEdit

Integer Storage:

Items: [101, 102, 103]

Item at index 1: 102

String Storage:

Items: [Alice, Bob, Charlie]

Item at index 2: Charlie

Double Storage:

Items: [99.99, 49.50, 150.75]

Item at index 0: 99.99